Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение
высшего образования
«Московский государственный технический университет имени Н.Э. Баумана
(национальный исследовательский университет)»
(МГТУ им. Н.Э. Баумана)

ФАКУЛЬТЕТ «ИНЖЕНЕРНЫЙ БИЗНЕС И МЕНЕДЖМЕНТ»
КАФЕДРА «ПРОМЫШЛЕННАЯ ЛОГИСТИКА»

ОТЧЕТ

Предмет: Разработка интернет приложений

Лабораторные работы №1,2

“Разработка веб-калькулятора в стилистике Kodik”

Студент: Сигорский Александр Евгеньевич

Группа: ИБМ3-54Б

Преподаватель: М.В. Бурмистрова

Москва 2026

Введение

В рамках лабораторных работ № 1 и № 2 разработан веб-калькулятор с интерфейсом, стилизованным под визуальный образ Kodik. Первая лабораторная работа посвящена HTML-разметке и CSS-оформлению, вторая — реализации логики на JavaScript и работе с событиями пользовательского интерфейса.

Проект выполнен в виде одного HTML-файла. CSS размещён внутри тега <style>, JavaScript — внутри тега <script>. Такой вариант выбран для удобства запуска и демонстрации: файл можно открыть локально в браузере без дополнительных зависимостей. Логотип также встроен в документ в формате data URI.

Основной экран оформлен как рабочая среда: слева расположена панель навигации по условным файлам проекта, в центре — калькулятор, справа — компактная информационная панель. Дополнительно реализованы история вычислений, горячие клавиши, простая командная панель и автоматическое уменьшение размера длинного значения на дисплее. Эти функции не меняют базовую схему работы калькулятора и остаются понятными для демонстрации на защите.

При разработке учитывались требования лабораторных работ: использование нескольких основных цветов выбранного сайта, hover-состояний, CSS-свойств для компоновки и отступов, а также реализация индивидуальной операции на JavaScript без дополнительных окон alert.

Цели и задачи работ

- Лабораторная работа № 1: изучить HTML и CSS, сверстать калькулятор и оформить его в стилистике выбранного сайта.
- Лабораторная работа № 2: реализовать работу калькулятора на JavaScript, обработку кнопок и индивидуальную операцию.

- Научиться получать HTML-элементы из JavaScript и обрабатывать события click и keydown.
- Обеспечить понятную связь между интерфейсом и кодом программы, чтобы работу можно было последовательно объяснить на защите.

1 Лабораторная работа № 1. Верстка калькулятора на HTML и CSS

1.1 Постановка задачи

Необходимо создать интерфейс калькулятора средствами HTML и CSS, подобрать 3–5 основных цветов по выбранному визуальному референсу, реализовать состояния hover и active, а также использовать дополнительные свойства оформления и компоновки. В данной работе референсом выступает Kodik: интерфейс выполнен в тёмной гамме с ярким зелёным акцентом и построен как компактная рабочая среда.

1.2 Структура HTML-документа

Верхний уровень документа состоит из стандартных элементов <!DOCTYPE html>, <html>, <head> и <body>. Внутри body размещён корневой контейнер .app, который делится на верхнюю панель, рабочую область и строку состояния. Центральная рабочая область содержит боковую навигацию, редактор с вкладками, страницы калькулятора, истории и README, а также правую информационную панель.

HTML-элемент	Назначение в работе
<header>	Верхняя панель с логотипом, поиском команд и быстрыми кнопками.
<main>	Основная рабочая область приложения.
<aside>	Боковая панель Explorer, панель иконок и правая панель Kodik Act.
<section>	Отдельные страницы калькулятора, истории и справки.
<button>	Клавиши калькулятора, вкладки, навигация и команды.
<input>	Поле ввода команд в командной панели.
	Служебные контейнеры для сетки, экрана и информационных

	блоков.

1.3 Цветовая палитра и оформление

Для интерфейса использована чёрно-зелёная палитра, визуально соответствующая логотипу и общему стилю Kodik. Основные цвета вынесены в CSS-переменные, поэтому их легко менять в одном месте.

Цвет	Код	Использование
Основной фон	#010402	Фон страницы и экрана калькулятора.
Фон панелей	#0A0F0B	Карточки, панели и рабочие блоки.
Зелёный акцент	#46BE4D	Главные кнопки, активные элементы и строка состояния.
Светлый зелёный	#64D56A	Hover-состояния и выделение результата.
Основной текст	#E7EBE7	Основной светлый текст интерфейса.
Границы	#202A22	Разделители и рамки элементов.

```
/* Общие настройки */
:root {
    --bg: #010402;
    --panel: #0b100c;
    --green: #46be4d;
    --green-light: #64d56a;
    --text: #e7ebe7;
    --line: #202a22;
}
```

1.4 Компоновка и CSS-свойства

Основная компоновка построена на CSS Grid и Flexbox. Grid используется для общей структуры рабочей области, клавиатуры калькулятора и таблицы истории. Flexbox применяется там, где элементы должны располагаться в одну строку и выравниваться по центру. Отступы задаются через padding и gap. Свойство margin в проекте не используется.

- display: grid — общая структура интерфейса и сетка клавиш;
- display: flex — верхняя панель, вкладки и строка состояния;
- padding — внутренние отступы контейнеров и кнопок;

- gap — расстояние между элементами сетки;
- border и border-radius — рамки и скругления;
- transition — плавная смена цвета и состояния;
- :hover и :active — визуальная обратная связь при наведении и нажатии;
- @media — адаптация интерфейса под меньшую ширину экрана.

```
.key:hover {
    background: #18211a;
    border-color: #344238;
}

.key:active {
    transform: translateY(1px);
    background: #1d281f;
}
```

На рисунке 1 показан основной экран разработанного интерфейса.

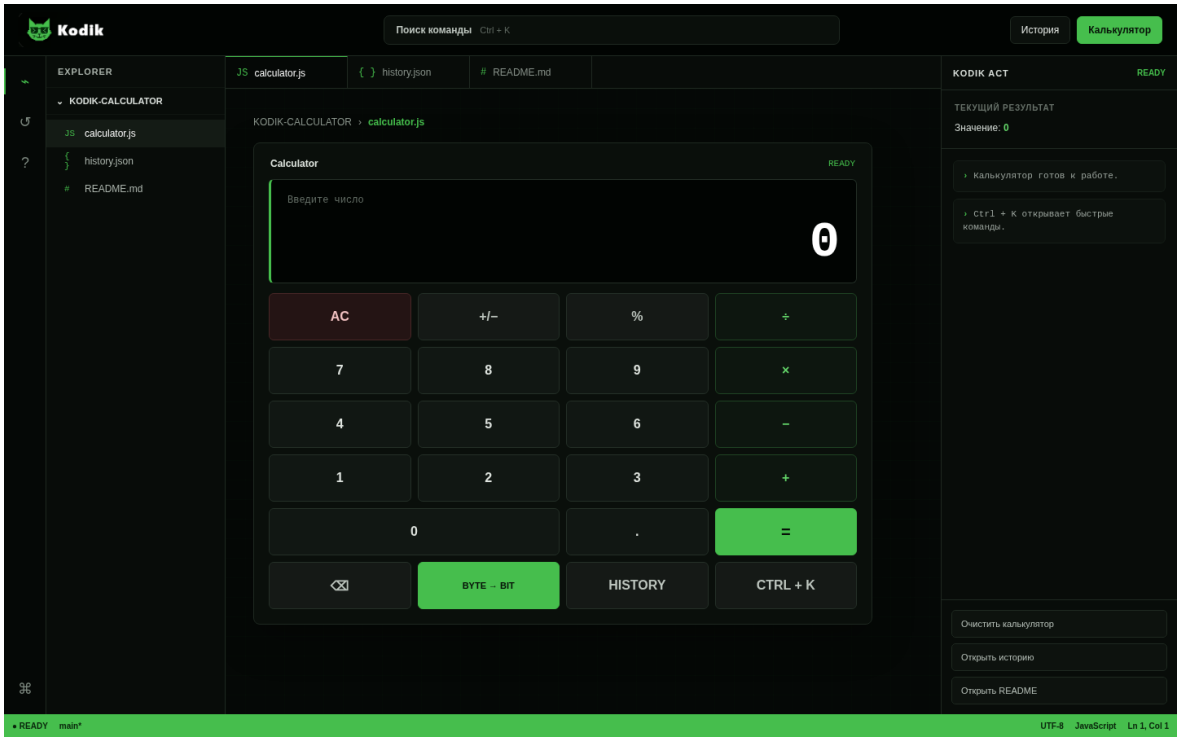


Рисунок 1 — Основной экран калькулятора в стилистике Kodik

1.5 Порядок демонстрации лабораторной работы № 1

1. Показать основные блоки HTML и объяснить назначение header, main, aside, section, button и div.
2. Показать CSS-переменные с основными цветами интерфейса.

3. Продемонстрировать hover и active на клавишах калькулятора и элементах навигации.
4. Объяснить использование Grid, Flexbox, padding, gap, border, border-radius и transition.
5. Показать адаптивность интерфейса при уменьшении ширины окна.

1.6 Ответы на контрольные вопросы

1.6.1 Основные составляющие Web

К основным составляющим Web относятся HTML, CSS и JavaScript. HTML определяет структуру и содержимое страницы, CSS отвечает за внешний вид, а JavaScript добавляет логику и интерактивность. Для передачи данных между клиентом и сервером используются протоколы HTTP/HTTPS, а ресурсы идентифицируются адресами URI/URL.

1.6.2 Что такое URI

URI (Uniform Resource Identifier) — унифицированный идентификатор ресурса. URI позволяет однозначно обозначить ресурс. URL является разновидностью URI и указывает, где ресурс находится и каким способом к нему обратиться. Типичная структура URL: схема://хост:порт/путь?параметры#фрагмент.

1.6.3 HTML-страница из заголовка и абзаца

```
<!DOCTYPE html>
<html lang="ru">
<head>
    <meta charset="UTF-8">
    <title>Моя страница</title>
</head>
<body>
    <h1>Заголовок</h1>
    <p>Абзац текста.</p>
</body>
</html>
```

1.6.4 Отличительные черты HTTP

HTTP — прикладной протокол взаимодействия клиента и сервера по модели запрос–ответ. Он не требует сохранения состояния между отдельными запросами: состояние приложения при необходимости поддерживается cookies, сессиями и другими механизмами. HTTP определяет методы запросов, например GET, POST, PUT и DELETE, заголовки и коды состояния ответов. HTTP/1.x использует текстовое представление стартовой строки и заголовков, а HTTP/2 и HTTP/3 применяют более современные механизмы передачи данных. HTTPS — это HTTP поверх защищённого соединения TLS.

2 Лабораторная работа № 2. Реализация логики на JavaScript

2.1 Постановка задачи

Необходимо сделать калькулятор рабочим с помощью JavaScript: реализовать ввод чисел, выбор арифметической операции, вычисление результата по кнопке «=», очистку и дополнительные действия. Также требуется добавить индивидуальную операцию, встроенную в общую логику калькулятора без использования alert. В итоговой версии дополнительно обеспечено корректное отображение длинных чисел внутри дисплея.

2.2 Хранение состояния калькулятора

Состояние калькулятора хранится в нескольких простых переменных. firstNumber содержит первое число, secondNumber — второе, selectedOperation — выбранную арифметическую операцию, а waitingSecondNumber показывает, начался ли ввод второго числа. Массив history используется для хранения последних вычислений.

```
let firstNumber = "";
let secondNumber = "";
let selectedOperation = null;
let waitingSecondNumber = false;
let history = [];
```

2.3 Ввод чисел и выбор операции

При нажатии цифровой кнопки вызывается функция inputDigit(). Если операция ещё не выбрана, цифра добавляется к firstNumber. После выбора операции ввод переключается на secondNumber. Повторный ввод десятичной точки блокируется, чтобы не формировать некорректное число.

Функция chooseOperation() сохраняет выбранный оператор и переводит калькулятор в режим ввода второго числа. Если пользователь уже ввёл оба числа и выбирает следующую операцию, текущее выражение сначала вычисляется.

2.4 Связь кнопки «=» с кодом программы

Кнопка «=» имеет атрибут data-action="equal". Для всех элементов с data-action установлен единый обработчик click. Внутри обработчика значение атрибута сравнивается с нужным действием. Для equal вызывается функция calculate(). Таким образом, связь между интерфейсом и вычислением задаётся через data-атрибут и обработчик события.

```
document.querySelectorAll("[data-action]").forEach(button => {
    button.addEventListener("click", () => {
        const action = button.dataset.action;

        if (action === "clear") clearCalculator();
        if (action === "equal") calculate();
        if (action === "bits") bytesToBits();
    });
});
```

2.5 Выполнение арифметической операции

Функция calculate() проверяет наличие двух чисел и выбранной операции, преобразует строки в Number и выполняет нужную ветку switch. После вычисления результат сохраняется в firstNumber, второе число и выбранная операция очищаются. Это позволяет сразу продолжить вычисления, используя полученный результат.

```
function calculate() {
    if (firstNumber === "" || secondNumber === "" || !selectedOperation) return;

    const a = Number(firstNumber);
    const b = Number(secondNumber);
    let result;

    switch (selectedOperation) {
        case "+": result = a + b; break;
        case "-": result = a - b; break;
        case "*": result = a * b; break;
        case "/":
            if (b === 0) {
                showError("Деление на ноль");
                return;
            }
            result = a / b;
            break;
    }

    firstNumber = formatNumber(result);
    secondNumber = "";
    selectedOperation = null;
    waitingSecondNumber = false;
}
```

На рисунке 2 показан результат сложения 2 + 3.

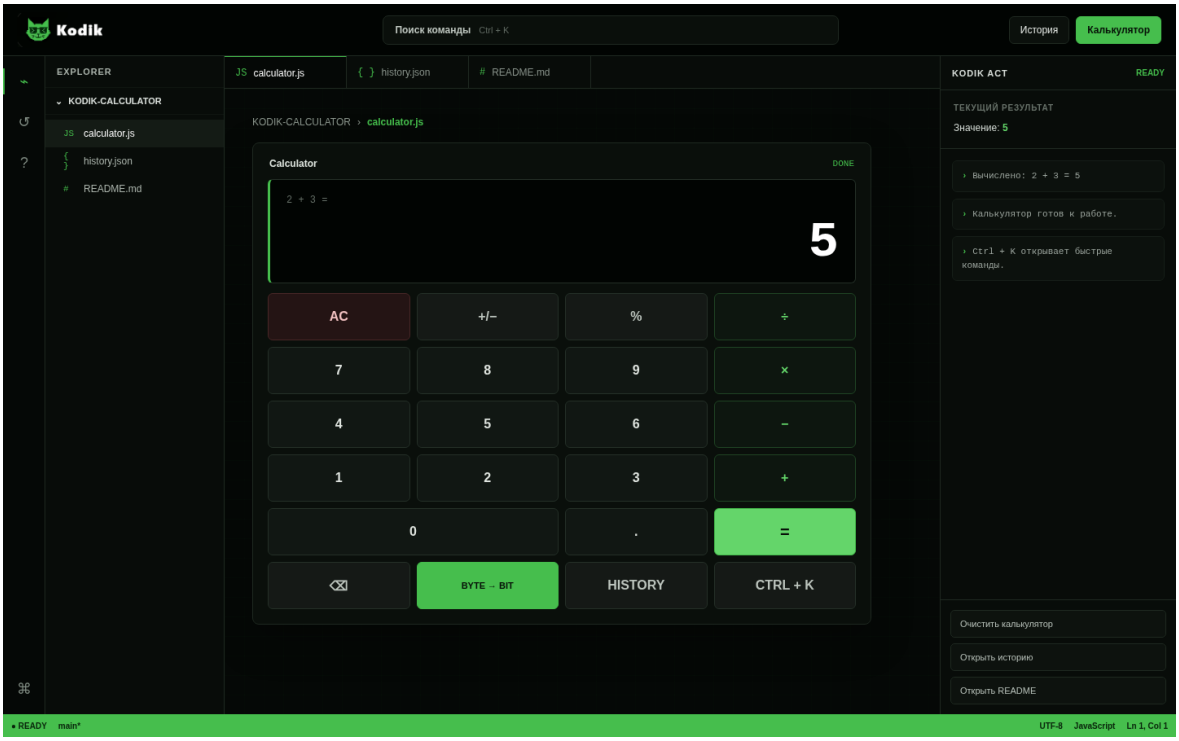


Рисунок 2 — Результат выполнения операции 2 + 3

2.6 Пошаговое выполнение сложения двух чисел

1. Пользователь нажимает кнопку 2. Обработчик получает data-digit="2" и вызывает inputDigit("2"). Значение firstNumber становится строкой "2".
2. Пользователь нажимает +. Вызывается chooseOperation("+"). В selectedOperation сохраняется "+", a waitingSecondNumber становится true.
3. Пользователь нажимает 3. Функция inputDigit("3") записывает "3" в secondNumber.
4. Пользователь нажимает =. Обработчик data-action вызывает calculate().
5. Строки "2" и "3" преобразуются к числам. В switch выбирается ветка case "+" и вычисляется 2 + 3.
6. Результат 5 преобразуется обратно в строку, сохраняется в firstNumber и выводится на экран.
7. Выражение 2 + 3 = 5 добавляется в историю вычислений.

2.7 Индивидуальная операция BYTE → BIT

В качестве индивидуальной операции реализован перевод байтов в биты. Операция встроена непосредственно в калькулятор: пользователь вводит количество байтов и нажимает кнопку BYTE → BIT. Отдельные окна не используются. Значение умножается на 8, результат выводится на основном экране и добавляется в историю.

```
function bytesToBits() {
    if (currentValue() === "0" && firstNumber === "") return;

    const bytes = Number(currentValue());
    const bits = bytes * 8;

    addHistory(bytes + " byte → bit", bits);
    firstNumber = formatNumber(bits);
    secondNumber = "";
    selectedOperation = null;
    waitingSecondNumber = false;

    expression.textContent = bytes + " bytes × 8 =";
    display.textContent = firstNumber;
}
```

Пример: 16 байт × 8 = 128 бит. Результат показан на рисунке 3.

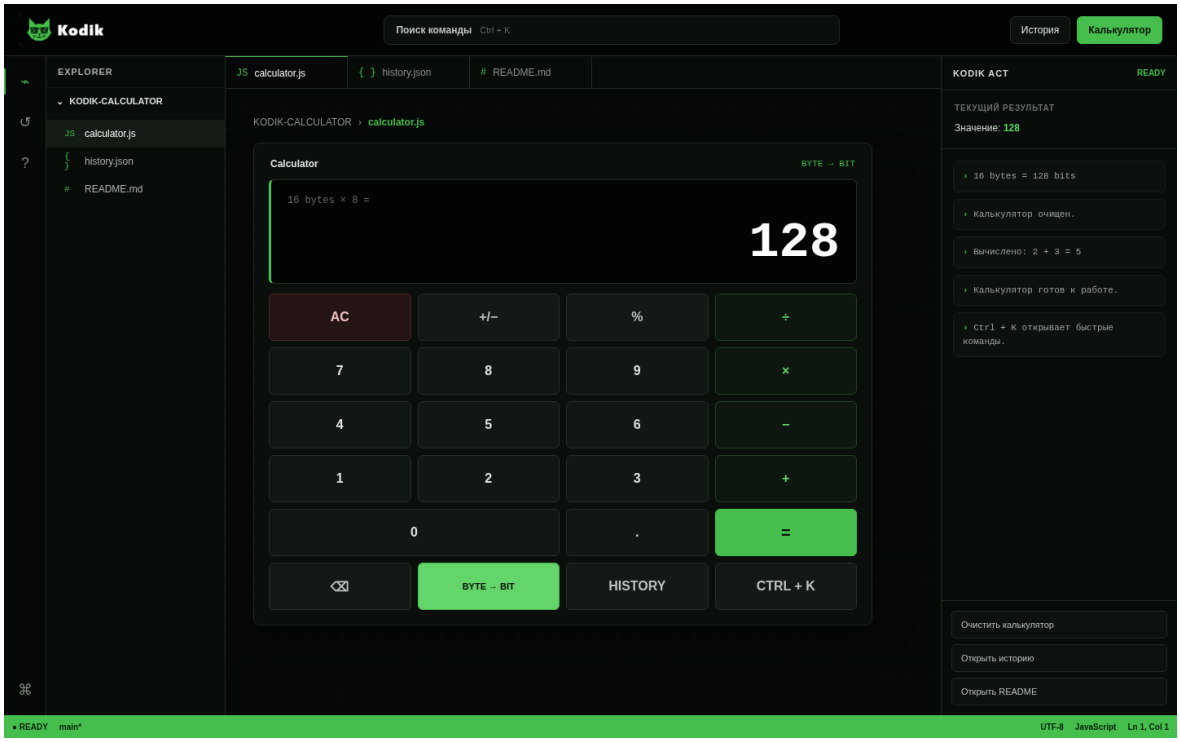


Рисунок 3 — Работа индивидуальной операции BYTE → BIT

2.8 История вычислений и дополнительные события

После каждого вычисления строка выражения и результат добавляются в массив history. В интерфейсе хранится не более десяти последних записей. Страница истории формируется динамически с помощью document.createElement() и innerHTML. Также реализовано управление с клавиатуры: цифры вводятся соответствующими клавишами, Enter выполняет вычисление, Backspace удаляет последний символ, Escape очищает калькулятор, Ctrl + K открывает командную панель.

На рисунке 4 показана страница истории после выполнения обычного вычисления и индивидуальной операции.

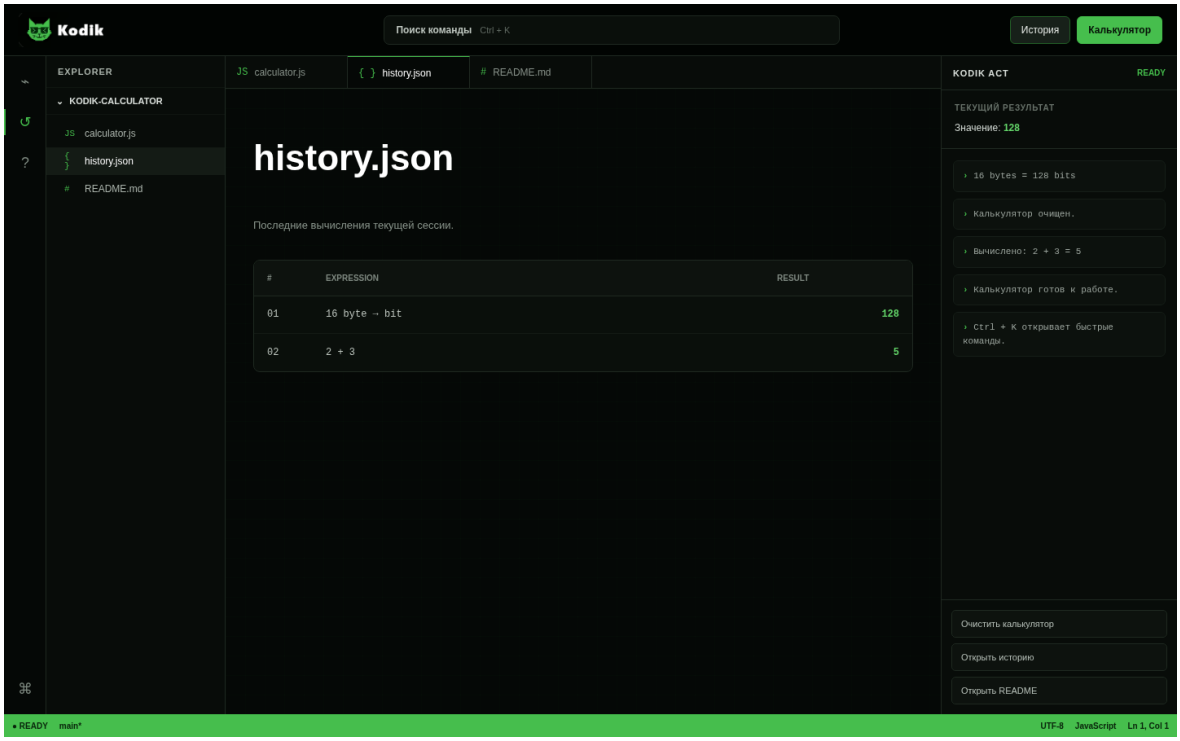


Рисунок 4 — История вычислений

2.9 Автоматическое уменьшение размера числа

Чтобы длинное число не выходило за границы дисплея, в итоговую версию добавлена функция fitDisplayText(). После каждого обновления значения она сравнивает фактическую ширину текста с шириной поля. Если строка не помещается, размер шрифта уменьшается по 2 px до тех пор, пока

число не окажется внутри дисплея. Минимальный размер ограничен 18 px, поэтому значение остаётся читаемым.

```
// Размер числа на экране
function fitDisplayText() {
    display.style.fontSize = "";

    let fontSize = parseFloat(getComputedStyle(display).fontSize);
    const minFontSize = 18;

    while (display.scrollWidth > display.clientWidth && fontSize > minFontSize) {
        fontSize -= 2;
        display.style.fontSize = fontSize + "px";
    }
}
```

Функция вызывается из updateDisplay(), после обычного вычисления и после операции BYTE → BIT. Кроме того, обработчик события resize запускает её при изменении ширины окна. Благодаря этому короткие значения отображаются крупным шрифтом, а длинные автоматически уменьшаются и стабильно помещаются в поле ввода.

```
window.addEventListener("resize", fitDisplayText);
```

2.10 Порядок демонстрации лабораторной работы № 2

1. Показать ввод двух чисел и выполнение одной из арифметических операций.
2. Показать HTML-кнопку «=», её data-action и обработчик JavaScript, вызывающий calculate().
3. Пошагово объяснить, как 2 + 3 проходит от нажатий пользователя до результата 5.
4. Продемонстрировать индивидуальную операцию BYTE → BIT на примере 16 байт → 128 бит.
5. Показать историю вычислений и обработку деления на ноль.

6. Ввести длинную последовательность цифр и показать, что размер значения автоматически уменьшается, не выходя за границы дисплея.

2.11 Ответы на контрольные вопросы

2.11.1 Как связана кнопка «=» с кодом лабораторной

Связь реализована через атрибут data-action="equal". После загрузки страницы JavaScript находит кнопки с data-action, назначает им обработчики click и при значении equal вызывает функцию calculate(). Именно calculate() получает два числа, определяет выбранную операцию и выводит результат.

2.11.2 Что происходит при сложении двух чисел

Сначала нажатия на цифровые кнопки формируют первое число как строку. После нажатия «+» этот оператор сохраняется в selectedOperation. Следующие цифры формируют secondNumber. Нажатие «=» вызывает calculate(), строки преобразуются в Number, выполняется сложение, результат сохраняется как новое firstNumber и отображается на экране.

2.11.3 Класс кнопки, выводящей количество нажатий

```
class ClickCounterButton {
    constructor(selector) {
        this.button = document.querySelector(selector);
        this.count = 0;

        this.button.addEventListener("click", () => {
            this.count++;
            this.button.textContent = "Нажатий: " + this.count;
        });
    }
}

new ClickCounterButton("#counterButton");
```

2.11.4 Поиск максимального элемента в массиве

```
function findMax(numbers) {
    if (numbers.length === 0) return undefined;

    let max = numbers[0];

    for (let i = 1; i < numbers.length; i++) {
        if (numbers[i] > max) {
            max = numbers[i];
        }
    }

    return max;
}
```

```
console.log(findMax([3, 7, 1, 9, 4])); // 9
```

Заключение

В ходе лабораторной работы № 1 был разработан интерфейс веб-калькулятора средствами HTML и CSS. Создана структура из верхней панели, навигации, центральной рабочей области, калькулятора и дополнительных страниц. Для оформления использована тёмная палитра с зелёным акцентом, реализованы hover- и active-состояния, адаптивная сетка и компоновка на Grid и Flexbox.

В ходе лабораторной работы № 2 реализована логика калькулятора на JavaScript. Поддерживаются базовые арифметические операции, проценты, смена знака, удаление последней цифры и обработка деления на ноль. Индивидуальная операция BYTE → BIT переводит байты в биты и встроена в основную логику без дополнительных окон. Дополнительно реализованы история вычислений, горячие клавиши и автоматическая подгонка размера шрифта: при длинном значении текст уменьшается до тех пор, пока полностью не поместится в поле дисплея.

Поставленные цели лабораторных работ достигнуты: интерфейс связан с программной логикой через DOM и обработчики событий, а основные действия можно последовательно продемонстрировать и объяснить на защите.

Список использованных источников

1. Репозиторий курса «Проектирование сетевых приложений». URL: https://github.com/iu5git/JavaScript
2. Методические указания к лабораторной работе № 1. URL: https://github.com/iu5git/JavaScript/tree/main/tutorials/lab1
3. Методические указания к лабораторной работе № 2. URL: https://github.com/iu5git/JavaScript/tree/main/tutorials/lab2
4. Kodik. Официальный сайт. URL: https://kodik.ru/

5. MDN Web Docs: HTML, CSS, JavaScript. URL: https://developer.mozilla.org/